渠凯铭

```cpp
#include <iostream>
#include <map>
#include <list>
#include <string>
#include <unordered_map>
#include <stdexcept>
#include <cassert>
#include <vector>
using namespace std;

// 订单类
class Order {
    friend class OrderBook;
private:
    int orderId;
    double price;
    char dir;
    int originalQuantity;
    int filledQuantity;
public:
    Order(int id,double p, char d, int q)
        : orderId(id), price(p), dir(d), originalQuantity(q), filledQuantity(0) {}
    int getFilled() const { return filledQuantity; }
    int getRemains() const { return originalQuantity - filledQuantity; }

};

class OrderBook {
private:
    // 买单：价格从高到低
    map<double, list<Order>, greater<double>> bids;
    // 卖单：价格从低到高
    map<double, list<Order>> asks;
    // 快速索引：ID -> <价格, 方向>
    unordered_map<int, pair<double, char>> orderIndex;

public:
    // 获取订单引用
    Order* getOrder(int id) {
        if (orderIndex.find(id) == orderIndex.end()) return nullptr;
        double p = orderIndex[id].first;
        char d = orderIndex[id].second;
        auto& list = (d == 'B') ? bids[p] : asks[p];
        for (auto& o : list) {
            if (o.orderId == id) return &o;
        }
        return nullptr;
    }

    // 插入订单：包含异常检查
    void insertOrder(int id,double price, char dir, int qty) {
        if (price <= 0 || qty <= 0)
            throw invalid_argument("价格和数量必须为正数");
```

```
        if (dir != 'B' && dir != 'S')
            throw invalid_argument("请使用B和S表示买单和卖单");
        if (orderIndex.count(id))
            throw runtime_error("订单编号重复");

        Order newOrder(id, price, dir, qty);
        if (dir == 'B') bids[price].emplace_back(newOrder);
        else asks[price].emplace_back(newOrder);

        orderIndex[id] = {price, dir};
        matchOrders();
    }

    // 取消订单
    void cancelOrder(int id) {
        if (orderIndex.find(id) == orderIndex.end()) {
            throw runtime_error("未发现请求取消的订单");
        }
        auto info = orderIndex[id];
        double p = info.first;// price
        char d = info.second;// dir

        if (d == 'B') {
            auto& list = bids[p];
            list.remove_if([id](const Order& o){ return o.orderId == id; });
            if (list.empty()) bids.erase(p);
        } else {
            auto& list = asks[p];
            list.remove_if([id](const Order& o){ return o.orderId == id; });
            if (list.empty()) asks.erase(p);
        }
        orderIndex.erase(id);
    }
    // 撮合逻辑
    void matchOrders() {
        while (!bids.empty() && !asks.empty()) {
            auto& bestBidList = bids.begin()->second;
            auto& bestAskList = asks.begin()->second;
            // 修改链表中对象
            Order& bid = bestBidList.front();
            Order& ask = bestAskList.front();
            //买价 < 卖价 则无法成交
            if (bid.price < ask.price) break;
            // 执行成交
            int tradeQty = min(bid.originalQuantity - bid.filledQuantity,
                               ask.originalQuantity - ask.filledQuantity);
            bid.filledQuantity += tradeQty;
            ask.filledQuantity += tradeQty;
            // 移除完全成交的订单
            if (bid.filledQuantity == bid.originalQuantity) {
                orderIndex.erase(bid.orderId);
                bestBidList.pop_front();
                if (bestBidList.empty()) bids.erase(bids.begin());
            }
            if (ask.filledQuantity == ask.originalQuantity) {
                orderIndex.erase(ask.orderId);
```

```
                bestAskList.pop_front();
                if (bestAskList.empty()) asks.erase(asks.begin());
            }
        }
    }
    // 显示深度
    void displayTop() const {
        // 自动推导 map 类型
        auto show = [](const auto& book,string label) {
            cout << label << ":\n";
            int n = 0;
            for (auto it = book.begin(); it != book.end() && n < 5; ++it, ++n) {
                int vol = 0;
                // 累加该价格下所有订单的剩余数量
                for (const auto& o : it->second) vol += (o.originalQuantity - o.filledQuantity);
                cout << " 价格 " << it->first << "剩余数量 " << vol << endl;
            }
        };
        cout << "\n--- 市场深度---\n";
        show(asks, "卖单");
        show(bids, "买单");
    }
};

void runSelfTests()
{
    cout << "自测数据: " << endl;
    // 测试用例 1: 题目场景
    {
        OrderBook ob;
        ob.insertOrder(1,10,'B',100);
        ob.insertOrder(2,11,'B',50);
        ob.insertOrder(4,9,'S',100);
        assert(ob.getOrder(2) == nullptr); // 买单2应该完全成交被移除
        const Order* o1 = ob.getOrder(1);
        assert(o1 != nullptr && o1->getFilled() == 50); // 买单1应该成交了50个
        assert(ob.getOrder(4) == nullptr); // 卖单4(100个)应该全部卖出
        ob.insertOrder(3,10,'S',50);
        assert(ob.getOrder(1) == nullptr);
        assert(ob.getOrder(3) == nullptr);
        cout << "[PASS] 价格优先实现" << endl;
    }
    // 测试用例 2: 异常处理测试
    {
        OrderBook ob;
        try {
            ob.insertOrder(1,-100,'B',10); // 非法价格
            assert(false);
        } catch (const invalid_argument& e) {}
        ob.insertOrder(1,100,'B',10);
        try {
            ob.insertOrder(1,100,'B',10); // 重复ID
            assert(false);
        } catch (const runtime_error& e) {}
        try {
            ob.cancelOrder(999); // 取消不存在的订单
```

```
            assert(false);
        } catch (const runtime_error& e) {}
        cout << "[PASS] 捕获“非法价格、重复订单ID和取消不存在订单”异常 实现"<< endl;
    }
    // 测试用例 3: 时间优先测试
    {
        OrderBook ob;
        ob.insertOrder(10,10,'B',10);
        ob.insertOrder(11,10,'B',10);
        ob.insertOrder(12,10,'S',10);
        assert(ob.getOrder(10) == nullptr);
        assert(ob.getOrder(11) != nullptr);
        cout << "[PASS] 时间优先实现" << endl;
    }
    cout << "全部自测数据通过\n" << endl;
}

int main()
{
    runSelfTests();
    OrderBook ob;
    ob.insertOrder(1,150.0,'B',100);
    ob.insertOrder(2,152.0,'B',50);
    ob.insertOrder(3,100.0,'B',50);
    ob.displayTop();
    ob.insertOrder(4,149.0,'S',100);
    ob.displayTop();
    ob.insertOrder(5,110,'S',100);
    ob.displayTop();
    return 0;
}
```